Outlier-Robust Kernel Learning

Eric Scheer, Jun Simons, Anu Arayl

Rensselaer Polytechnic Institute

April 24th, 2026

Outline

1. Problem Overview
2. Related Work
3. Kernel Learning
4. Robust Learning
5. Results
6. Future Work

Problem Overview

Overview

In this project, we examine the use of iterative methods to perform supervised learning on data that has been corrupted, potentially adversarially.

Specifically, we apply work done by Rathnashyam & Gittens (2024) in developing a robust algorithm for learning a single neuron model to the problem of robust kernel learning.

Learning Under Adversarial Corruption

We study supervised regression when the training data can be **corrupted** before it is given to the learner.

Setting:

- We have labled training samples $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$
- An adversary is allowed to inspect the full dataset
- Then it arbitrarily modifies an ϵ fraction of samples
- Both **covariates** and **labels** may be corrupted

Goal: learn a predictor that fits the original signal (before corruption) despite the outliers.

Strong ϵ-Contamination Model

Definition (Strong ϵ-contamination model)

Given n clean samples drawn i.i.d. from a distribution $\mathcal{D}$, an adversary may inspect the full sample and arbitrarily modify ϵn points before training.

Our labels satisfy:

$$y_i = f(\mathbf{w}_*^\top \mathbf{x}_i) + \xi_i, \qquad \xi_i \sim \mathcal{N}(0, \sigma^2),$$

where $\mathbf{w}_*$ is a vector of ground truth weights. The clean covariates $\{\mathbf{x}_i\}_{i=1}^n$ are assumed to come from a centered sub-Gaussian distribution with known covariance matrix Σ.

Standard least-squares or Kernel Ridge Regression (KRR) is likely to fit the corrupted points and give a bad prediction.

Related Work

Related Work

- Estimation tasks under potentially adversarial corruption have been well-studied. Early work including Huber's contamination model provide statistical frameworks to model datasets that contain both clean data points and arbitrary outliers (Huber, 1964).
- Bhatia et al. (2015) showed that under a linear model with dense, bounded noise $\boldsymbol{\xi}$ and unbounded, adversarial label corruption $\mathbf{b}$, Iterative Thresholding Gradient Descent (ITGD) returns a $\mathcal{O}\left(\frac{\|\boldsymbol{\xi}\|}{\sqrt{n}}\right)$-approximate iterate after $T = \Omega\left(\log \frac{\|\mathbf{b}\|}{\sqrt{n}\varepsilon}\right)$ iterations.
- Awasthi et al. (2022) used iterative thresholding with full solves to find $\mathcal{O}\left(\sigma\epsilon \log(1/\epsilon)\right)$-approximate iterate after $T = \Omega(1/\epsilon^2)$ iterations with high probability, where σ is the standard deviation of our Gaussian noise.
- More recently, Rathnashyam & Gittens (2024) used ITGD and covariate filtering (Dong et al., 2019) for learning single-neuron models with Gaussian noise with similar approximation bounds.

Why Kernel Methods?

Prior robust iterative methods were developed for learning a linear or single neuron model.

We want to extend this idea to **kernel methods** so we can:

- keep robustness to corrupted samples
- extend beyond linear predictors
- capture nonlinear structure using the kernel trick

Main question:
Can iterative filtering and thresholding be combined with kernel regression to obtain a more robust nonlinear learner?

Kernel Learning

Kernel Methods - Overview

Suppose the relationship between the covariates and labels is not well modeled by a linear predictor.

We can mitigate by peforming a feature transform from the original feature space $\mathcal{X}$ to a new feature space $\mathcal{Z}$ with a map

$$\phi : \mathcal{X} \to \mathcal{Z}$$

For example, if ϕ is the second-order polynomial transform, then

$$\phi\left(\begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}\right) = \begin{bmatrix} x_1^2 \\ x_1 x_2 \\ \vdots \\ x_d x_{d-1} \\ x_d^2 \end{bmatrix}$$

Problem: More complicated choices of ϕ can drastically increase the number of features.

Kernel Methods - Overview

Kernel trick: if the algorithm only needs inner products

$$\phi(x_i)^\top \phi(x_j),$$

we can replace them with a kernel function

$$\kappa(x_i, x_j) = \phi(x_i)^\top \phi(x_j),$$

without ever explicitly computing $\phi(x)$.

This lets us efficiently fit **nonlinear decision rules** using linear models in the transformed space.

Standard Regularized Learning Problem

We begin with the regularized empirical risk minimization problem with loss function ℓ and activation function f:

$$\hat{\mathbf{w}} = \arg\min_{w} \ \frac{1}{n}\sum_{i=1}^{n} \ell\Big(f(\mathbf{x}_i^\top \mathbf{w}), y_i\Big) + \frac{\lambda}{2}\|w\|_2^2.$$

With mean squared error (MSE) loss with identity activation, this becomes standard ridge regression:

$$\hat{\mathbf{w}} = \arg\min_{w} J(\mathbf{w}) = \arg\min_{w} \frac{1}{n}\sum_{i=1}^{n}(\mathbf{x}_i^\top \mathbf{w} - y_i)^2 + \frac{\lambda}{2}\|w\|_2^2.$$

Issue: Computing this objective requires computing a potentially high-dimensional feature transform.

Kernelized Formulation

Let $X \in \mathbb{R}^{n \times d}$ be our row-wise data matrix.

Since our objective J is convex, the minimizer $\hat{\mathbf{w}}$ must satisfy $\nabla J(\hat{\mathbf{w}}) = \mathbf{0}$. This tells us that $\hat{\mathbf{w}}$ must lie in the span of the datapoints. Equivalently, for some vector $\boldsymbol{\alpha} \in \mathbb{R}^n$,

$$\hat{\mathbf{w}} = X^\top \boldsymbol{\alpha}^\star.$$

Substituting gives the kernelized objective

$$\begin{aligned}
\boldsymbol{\alpha}^\star &= \min_{\boldsymbol{\alpha} \in \mathbb{R}^n} \frac{1}{n} \|XX^\top \boldsymbol{\alpha} - \mathbf{y}\|_2^2 + \frac{\lambda}{2} \|X^\top \boldsymbol{\alpha}\|_2^2 \\
&= \min_{\boldsymbol{\alpha} \in \mathbb{R}^n} \frac{1}{n} \|K\boldsymbol{\alpha} - \mathbf{y}\|_2^2 + \frac{\lambda}{2} \boldsymbol{\alpha}^\top K \boldsymbol{\alpha}
\end{aligned}$$

where K is our kernel matrix defined by

$$K_{ij} = \mathbf{x}_i^\top \mathbf{x}_j$$

The Kernel Trick

Since the entries in our kernel matrix only depend on inner products, we can redefine our kernel matrix as

$$K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

This allows us to learn with high, or even infinite dimensional $\mathcal{Z}$-spaces without needing to tranform vectors into this space.

Examples of kernels:

- Linear: $\kappa(x_i, x_j) = \mathbf{x}_i^\top x_j$
- Polynomial
- RBF / Gaussian

Robust Learning

Approach to Outlier Robustness

We do not want every sample to influence the update equally.

The ITGD approach combines two ideas:

1. **Covariate filtering (Dong et al., 2019):** assign weights $\omega_i \in [0, 1]$ to each label to mitigate covariate corruption.
2. **Iterative hard thresholding (Legendre, 1805):** at each gradient descent step, keep only the $(1 - \epsilon)n$ samples with the smallest residuals to mitigate label corruption.

Iterative Hard Thresholding

Definition (Hard Thresholding Operator (Bhatia et al., 2015))

For any vector $\mathbf{v} \in \mathbb{R}^n$, let $\sigma_{\mathbf{v}} \in S_n$ be a permutation that orders the entries of $\mathbf{v}$ in increasing order of magnitude, i.e., $|\mathbf{v}_{\sigma_{\mathbf{v}}(1)}| \leq |\mathbf{v}_{\sigma_{\mathbf{v}}(2)}| \leq \cdots \leq |\mathbf{v}_{\sigma_{\mathbf{v}}(n)}|$. Then, for any $k \leq n$, the hard thresholding operator is defined as the index set

$$\mathrm{HT}(\mathbf{v}; k) = \left\{ i \in [n] : -\sigma_{\mathbf{v}}^{-1}(i) > n - k \right\}, \tag{HT}$$

i.e., it selects the indices corresponding to the k largest-magnitude entries of $\mathbf{v}$.

For a current iterate $\boldsymbol{\alpha}_t$, define the residual loss of each sample by

$$r_i^{(t)} = \left((K\boldsymbol{\alpha}_t)_i - y_i \right)^2.$$

Then select the $(1-\epsilon)n$ smallest-loss points:

$$S_t = \mathrm{HT}(\mathbf{r}^{(t)}, (1-\epsilon)n).$$

Finally update only using those points:

$$\boldsymbol{\alpha}_{t+1} = \boldsymbol{\alpha}_t - \eta \sum_{i \in S_t} \omega_i \left((K\boldsymbol{\alpha}_t)_i - y_i \right) K_i.$$

Algorithm

Algorithm 1: Kernel Ridge Regression with Covariate Filtering and Iterative Hard Thresholding

$D = \{(\Sigma^{-1/2}\mathbf{x}_1, y_1), \ldots, (\Sigma^{-1/2}\mathbf{x}_n, y_n)\}$
$K_{i,j} = \mathbf{x}_i^\top \mathbf{x}_j$ for all $i, j \in \{1, \ldots, n\}$
$\boldsymbol{\omega} = \textsc{CovariateFiltering}(D, \epsilon)$ ▷ Algorithm 4 of (Dong et al., 2019)
for $t = 0, \ldots, T-1$ **do**
    $r_i^{(t)} \leftarrow ((K\boldsymbol{\alpha})_i - y_i)^2$ for all $i \in \{1, \ldots, n\}$ ▷ Calculate $\mathcal{L}$ for each sample
    $S_t \leftarrow \mathrm{HT}(\mathbf{r}^{(t)}, (1-\epsilon)n)$ ▷ Hard thresholding
    $\boldsymbol{\alpha}_{t+1} \leftarrow \boldsymbol{\alpha}_t - \eta \sum_{i \in S_t}((K\boldsymbol{\alpha})_i - y_i) \cdot \omega_i K_i$ ▷ Weighted gradient descent update
end for
return: $\boldsymbol{\alpha}_t$

What We Evaluate

We compare:

- Linear ITGD
- Kernel ITGD with linear / polynomial / RBF kernels
- Standard Kernel Ridge Regression
- Huberized kernel baselines
- OLS on corrupted data
- Inlier oracle baselines

We ask:

- Does thresholding improve robustness under corruption?
- How do different forms of corruption impact results?
- Does kernalization help under misspecification?

Results

Results

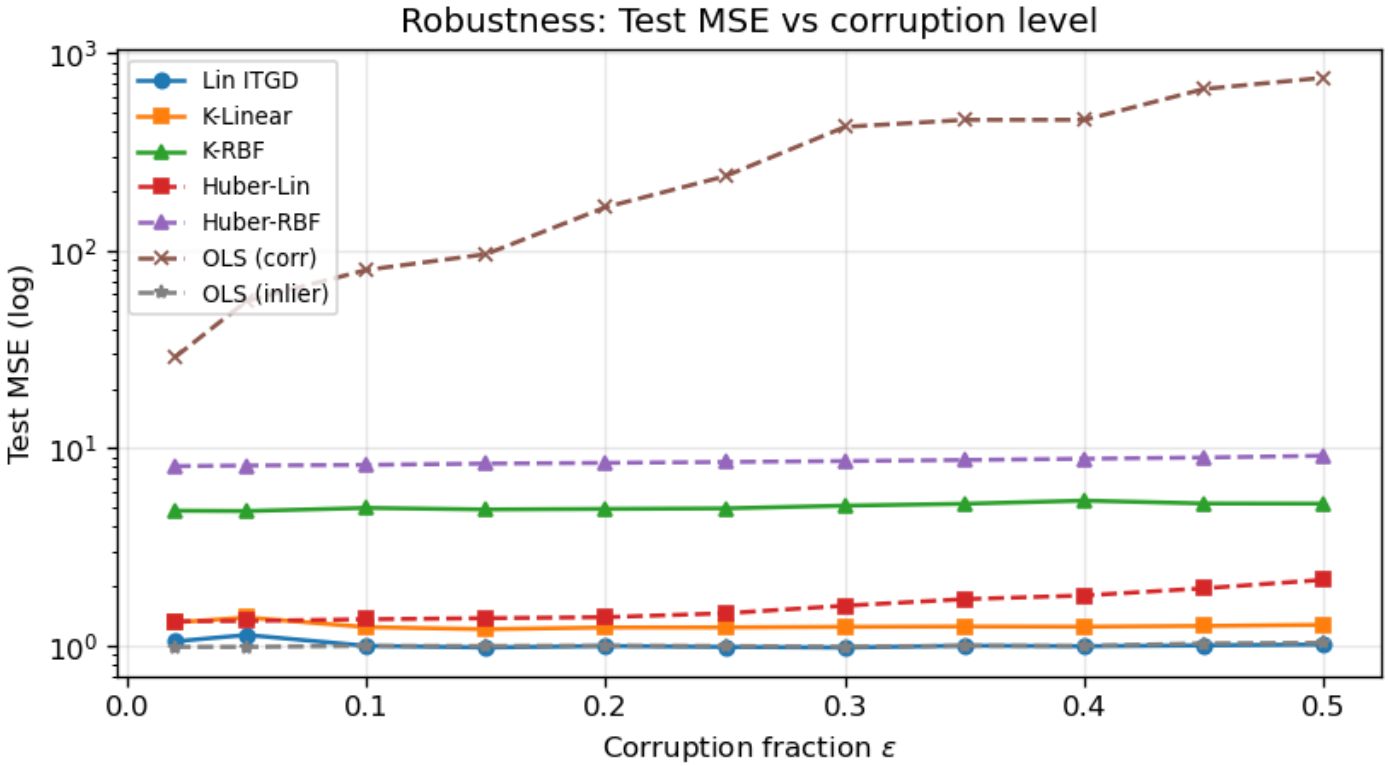

Figure: Comparison under increased additive gaussian noise

Results

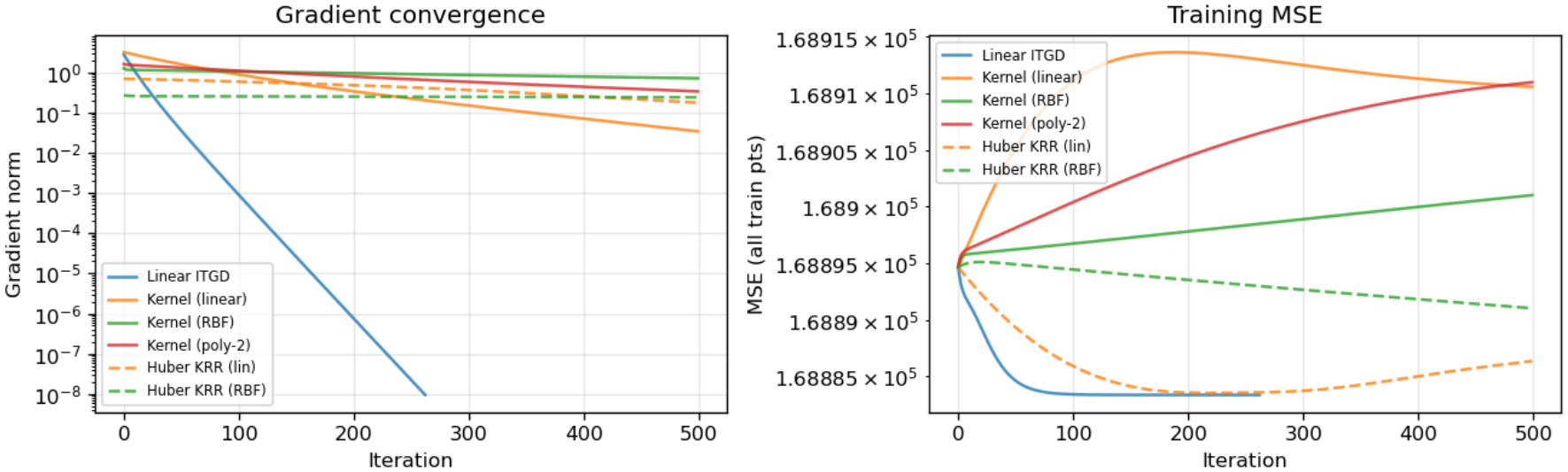

Figure: Gradient Convergence, $\epsilon = 0.15$

Results

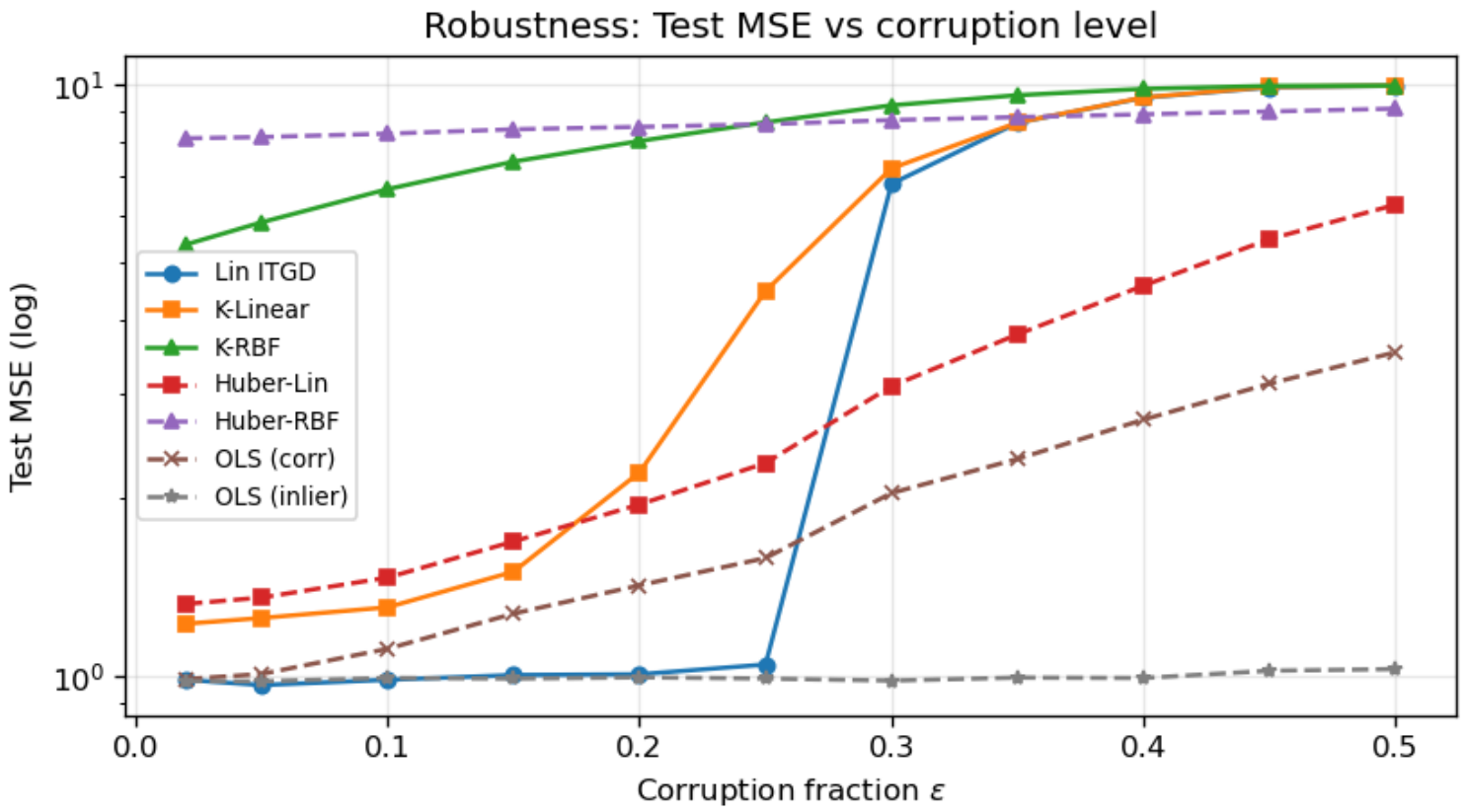

Figure: *Test MSE versus corruption level when an ϵ fraction of labels are set to zero. Linear ITGD performs better*

Results

Table: Performance summary on leaky-ReLU-generated data with additive Gaussian label corruption.

Method	**Test MSE range**
Lin ITGD (leaky, correct model)	$0.2355 \to 0.2459$
OLS (inlier oracle)	$0.6964 \to 0.7704$
K-RBF ITGD (identity)	$0.7956 \to 0.8914$
K-RBF ITGD (leaky)	$0.7887 \to 0.8981$
Huber-RBF (identity)	$0.9565 \to 1.1060$
Lin ITGD (identity, misspecified)	$1.1122 \to 1.1343$
K-Linear ITGD (leaky)	$1.5395 \to 1.5662$
OLS (corrupted)	$1.6164 \to 4.1233$
K-Linear ITGD (identity)	$1.7069 \to 1.7485$
KRR (corrupted, linear)	$3.1814 \to 8.7565$

Future Work

Future Work

- Investigation into alternative kernels, and whether similar outlier robustness techniques are relevant
- Looking into statistical breakpoints where methods break down
- Further examination of different forms of corruption, such as smarter adversaries that aim to cause maximum harm
- Alternative robustness methods

References

Pranjal Awasthi, Abhimanyu Das, Weihao Kong, and Rajat Sen. Trimmed maximum likelihood estimation for robust learning in generalized linear models, 2022. URL `https://arxiv.org/abs/2206.04777`.

Kush Bhatia, Prateek Jain, and Purushottam Kar. Robust regression via hard thresholding. *CoRR*, abs/1506.02428, 2015. URL `http://arxiv.org/abs/1506.02428`.

Yihe Dong, Samuel B. Hopkins, and Jerry Li. Quantum entropy scoring for fast robust mean estimation and improved outlier detection, 2019. URL `https://arxiv.org/abs/1906.11366`.

Peter J. Huber. Robust estimation of a location parameter. *Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.

Adrien-Marie Legendre. On the method of least squares. In D. E. Smith (ed.), *A Source Book in Mathematics*, pp. 576–579. Dover Publications, New York, 1805. Translated from the French (original work published 1805).

Arvind Rathnashyam and Alex Gittens. Iterative thresholding for non-linear